ğrencibilgiLab 05 – Hafta 5Qt Pencere ve Düzen Yönetimi

Lab Hakkında

Bu laboratuvar çalışmasında Qt6'nın düzen yöneticilerini (layout managers), `QMainWindow` yapısını, diyalog pencerelerini ve bunların OBYS projesinde nasıl bir araya getirileceğini uygulamalı olarak öğreneceksiniz. Her çalışma bölümünde önce örnek kodu inceleyip çalıştırın, ardından görevleri tamamlayın.
Süre: 2 saat **Gereksinimler:** Qt6, CMake 3.16+, C++20 destekli derleyici

Qt Projesi Derleme

Tüm çalışmalarda aşağıdaki CMake yapısını kullanın:

```
cmake_minimum_required(VERSION 3.16)
project(LabCalisma LANGUAGES CXX)
set(CMAKE_CXX_STANDARD 20)
find_package(Qt6 REQUIRED COMPONENTS Widgets)
qt_standard_project_setup()
qt_add_executable(app main.cpp)
target_link_libraries(app PRIVATE Qt6::Widgets)
```

Derleme komutları:

```
mkdir build && cd build
cmake ..
cmake --build .
```

1 Çalışma 1: Düzen Yöneticileri (30 dk)

Ön Bilgi

Qt'da widget'ları elle konumlandırmak (`setGeometry()`) yerine **düzen yöneticileri** (layout managers) kullanılır. Düzen yöneticileri pencere boyutu değiştiğinde widget'ları otomatik olarak yeniden konumlandırır. Temel düzen yöneticileri:

- `QHBoxLayout` – Widget'ları yatay (soldan sağa) sıralar
- `QVBoxLayout` – Widget'ları dikey (yukarıdan aşağıya) sıralar
- `QGridLayout` – Widget'ları satır/sütun ızgarasına yerleştirir
- `QFormLayout` – Etiket-alan çiftleri oluşturur

Düzenler iç içe yerleştirilerek (nested layouts) karmaşık arayüzler oluşturulabilir.

Aşağıdaki başlangıç kodunu `main.cpp` dosyasına yazın. Önce pencere ve dikey düzeni oluşturuyoruz:

```
#include <QApplication>
#include <QWidget>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QLabel>
#include <QPushButton>
#include <QLineEdit>
#include <QGroupBox>

int main(int argc, char* argv[]) {
```

```
    QApplication app(argc, argv);

    QWidget pencere;
    pencere.setWindowTitle("OBYS - Duzen Yonetimi");
    pencere.resize(500, 400);

    // Ana duzen: dikey
    auto* anaDuzen = new QVBoxLayout(&pencere);

    // Baslik
    auto* baslik = new QLabel("OBYS  renci  Arama");
    baslik->setStyleSheet(
        "font-size: 16px; font-weight: bold;"
        " color: #008080;");
    baslik->setAlignment(Qt::AlignCenter);
    anaDuzen->addWidget(baslik);
```

Şimdi yatay düzen ile arama çubuğu oluşturuyoruz. `QHBoxLayout` widget'ları yan yana yerleştirir:

```
    // Arama cubugu: yatay duzen
    auto* aramaDuzen = new QHBoxLayout;
    auto* aramaAlani = new QLineEdit;
    aramaAlani->setPlaceholderText(
        "  renci  adi veya numarasi...");
    auto* araButon = new QPushButton("Ara");
    auto* temizleButon = new QPushButton("Temizle");
    aramaDuzen->addWidget(aramaAlani);
    aramaDuzen->addWidget(araButon);
    aramaDuzen->addWidget(temizleButon);
    anaDuzen->addLayout(aramaDuzen);
```

Son olarak `QGroupBox` ile gruplandırılmış iç içe düzen ekliyoruz:

```
    // Sonuc grubu: QGroupBox i inde dikey duzen
    auto* sonucGrubu = new QGroupBox("Arama Sonuclari");
    auto* sonucDuzen = new QVBoxLayout;
    sonucDuzen->addWidget(new QLabel("Sonuc 1: Ali Yilmaz"));
    sonucDuzen->addWidget(new QLabel("Sonuc 2: Ayse Kaya"));
    sonucDuzen->addWidget(new QLabel("Sonuc 3: Mehmet Demir"));
    sonucGrubu->setLayout(sonucDuzen);
    anaDuzen->addWidget(sonucGrubu);

    // Alt bilgi: yatay duzen
    auto* altDuzen = new QHBoxLayout;
    altDuzen->addWidget(new QLabel("Toplam: 3 sonuc"));
    altDuzen->addStretch();
    altDuzen->addWidget(new QPushButton("Disa Aktar"));
    anaDuzen->addLayout(altDuzen);

    pencere.show();
    return app.exec();
}
```

`addStretch()` ile Boşluk Ekleme

`addStretch()` düzene esnek boşluk ekler. Bu sayede widget'lar birbirine yapışmaz ve düzen daha dengeli görünür. Örneğin yatay düzende soldaki etiketi sola, sağdaki butonu sağa yaslamak için aralarına `addStretch()` ekleyebilirsiniz.

Görev 1.1 – İç İçe Düzen Oluşturma (10 dk)

Başlangıç kodunu genişleterek:

a) Arama sonuçları grubunun altına yeni bir `QGroupBox` ekleyin: `"Filtreler"`.

b) Bu grubun içinde `QHBoxLayout` kullanarak 3 adet `QPushButton` yerleştirin: `"Bölüm"`, `"Sinif"`, `"Yariyil"`.

c) Filtre butonlarının arasına `addStretch()` ile eşit boşluk ekleyin.

Görev 1.2 – QGridLayout ile Hesap Makinesi Düzeni (20 dk)

`QGridLayout` kullanarak basit bir hesap makinesi düzeni oluşturun:

a) Yeni bir pencere oluşturun: `"OBYS - Not Hesaplayici"`.

b) Üst kısma bir `QLineEdit` ekleyin (ekran alanı).

c) Altına `QGridLayout` ile 4x4 buton ızgarası oluşturun:

```
auto* grid = new QGridLayout;
// addWidget(widget, satir, sutun)
grid->addWidget(new QPushButton("7"), 0, 0);
grid->addWidget(new QPushButton("8"), 0, 1);
grid->addWidget(new QPushButton("9"), 0, 2);
grid->addWidget(new QPushButton("/"), 0, 3);
```

d) Kalan satırları da doldurun: 4-5-6-*, 1-2-3–, 0-.-=+

e) Butonların minimum boyutunu ayarlayın:

```
buton->setMinimumSize(60, 40);
```

Dikkat: Düzen İç İçe Geçirme

Bir düzen yöneticisini başka bir düzene eklerken `addLayout()` kullanılır, `addWidget()` değil. `addWidget()` yalnızca widget'lar için kullanılır. Karıştırmak derleme hatasına yol açar.

2 Çalışma 2: QMainWindow ve Menüler (30 dk)

Ön Bilgi

`QMainWindow` profesyonel uygulamalar için hazır bir iskelet sunar. Menü çubuğu (menu bar), araç çubukları (toolbars), durum çubuğu (status bar) ve merkezi widget alanı içerir. Menü öğeleri `QAction` nesneleriyle temsil edilir ve sinyal/slot ile bağlanır.
`QMainWindow` yapısı:

- `menuBar()` – Menü çubuğuna erişim
- `addToolBar()` – Araç çubuğu ekleme
- `statusBar()` – Durum çubuğuna erişim
- `setCentralWidget()` – Merkezi widget belirleme

Aşağıdaki kodu `main.cpp` dosyasına yazın. Önce `QMainWindow` ve menü çubuğunu oluşturuyoruz:

```
#include <QApplication>
#include <QMainWindow>
#include <QMenuBar>
#include <QMenu>
#include <QAction>
#include <QToolBar>
#include <QStatusBar>
#include <QTextEdit>
#include <QDebug>

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    QMainWindow anaPencere;
    anaPencere.setWindowTitle("OBYS - Ana Pencere");
    anaPencere.resize(700, 500);

    // Merkezi widget: metin d zenleyici
    auto* editor = new QTextEdit;
    editor->setPlaceholderText(
        "OBYS kayitlari burada g r nt lenir...");
    anaPencere.setCentralWidget(editor);
```

Şimdi `QMenu` ve `QAction` ile menü çubuğunu dolduruyoruz:

```
    // --- Menu Cubugu ---
    auto* dosyaMenu = anaPencere.menuBar()->addMenu("&Dosya");

    auto* yeniAksiyon = new QAction("&Yeni", &anaPencere);
    yeniAksiyon->setShortcut(QKeySequence("Ctrl+N"));
    dosyaMenu->addAction(yeniAksiyon);

    auto* kaydetAksiyon = new QAction("&Kaydet", &anaPencere);
    kaydetAksiyon->setShortcut(QKeySequence("Ctrl+S"));
    dosyaMenu->addAction(kaydetAksiyon);

    dosyaMenu->addSeparator();

    auto* cikisAksiyon = new QAction("C&ikis", &anaPencere);
    cikisAksiyon->setShortcut(QKeySequence("Ctrl+Q"));
```

```
    dosyaMenu->addAction(cikisAksiyon);
```

Araç çubuğu ve durum çubuğu ekleyip sinyal/slot bağlantılarını kuruyoruz:

```
    // --- Arac Cubugu ---
    auto* aracCubugu = anaPencere.addToolBar("Ana Araclar");
    aracCubugu->addAction(yeniAksiyon);
    aracCubugu->addAction(kaydetAksiyon);

    // --- Durum Cubugu ---
    anaPencere.statusBar()->showMessage("Hazir", 3000);

    // --- Sinyal/Slot Baglantilari ---
    QObject::connect(yeniAksiyon, &QAction::triggered,
        [editor, &anaPencere]() {
            editor->clear();
            anaPencere.statusBar()->showMessage(
                "Yeni belge olu turuldu", 2000);
        });

    QObject::connect(kaydetAksiyon, &QAction::triggered,
        [&anaPencere]() {
            anaPencere.statusBar()->showMessage(
                "Kaydedildi!", 2000);
            qDebug() << "Kaydet tiklandi";
        });

    QObject::connect(cikisAksiyon, &QAction::triggered,
                     &app, &QApplication::quit);

    anaPencere.show();
    return app.exec();
}
```

Klavye Kısayolları ve & İşareti

Menü adındaki `&` karakteri bir alt çizgi kısayolu tanımlar: `"&Dosya"` Alt+D ile açılır. `QAction` üzerinde `setShortcut()` ile ek kısayol atanabilir. `QKeySequence` standart kısayolları da destekler: `QKeySequence::Save`, `QKeySequence::New` vb.

Görev 2.1 – Temel Menü Oluşturma (10 dk)

Başlangıç kodunu genişleterek:

a) `"Görünüm"` adında yeni bir menü ekleyin.

b) Bu menüye `"Tam Ekran"` aksiyonu ekleyin (kısayol: `F11`).

c) Tam Ekran aksiyonuna tıklandığında `showFullScreen()` veya `showNormal()` arasında geçiş yapın:

```
bool tamEkran = false;
QObject::connect(tamEkranAksiyon, &QAction::triggered,
    [&anaPencere, &tamEkran]() {
        tamEkran = !tamEkran;
        if (tamEkran) anaPencere.showFullScreen();
        else anaPencere.showNormal();
    });
```

d) Durum çubuğuna pencere modunu yazdırın.

Görev 2.2 – Düzenle Menüsü (20 dk)

`"Duzenle"` menüsü oluşturun:

a) Dosya ve Görünüm menüleri arasına `"Duzenle"` menüsü ekleyin.

b) Aşağıdaki aksiyonları oluşturun:

- `"Kes"` – kısayol: `Ctrl+X`
- `"Kopyala"` – kısayol: `Ctrl+C`
- `"Yapistir"` – kısayol: `Ctrl+V`

c) Her aksiyonu `QTextEdit`'in ilgili slot'larına bağlayın:

```
QObject::connect(kesAksiyon, &QAction::triggered,
                 editor, &QTextEdit::cut);
```

d) Kes ve Kopyala aksiyonlarını araç çubuğuna da ekleyin.

e) Her işlemden sonra durum çubuğunda bilgi mesajı gösterin.

Dikkat: QMainWindow ve setCentralWidget

`QMainWindow` üzerine doğrudan layout atanmaz. Bunun yerine `setCentralWidget()` ile merkezi bir widget belirlenir. Eğer birden fazla widget gerekiyorsa, bir `QWidget` kapsayıcı oluşturup ona layout atayın ve bunu merkezi widget olarak ayarlayın.

3 Çalışma 3: Diyalog Pencereleri (30 dk)

Ön Bilgi

Diyalog pencereleri kullanıcıdan bilgi almak veya bilgi göstermek için kullanılır. Qt'da iki tür diyalog vardır:

- **Modal diyalog**: Kullanıcı diyaloğu kapatmadan ana pencereyle etkileşime geçemez (`exec()`).
- **Modeless diyalog**: Ana pencereyle eş zamanlı kullanılabilir (`show()`).

Qt hazır diyaloglar sunar: `QMessageBox` (mesaj gösterme), `QInputDialog` (giriş alma), `QFileDialog` (dosya seçme), `QColorDialog` (renk seçme).

Aşağıdaki kodu `main.cpp` dosyasına yazın. Önce pencere ve butonları oluşturuyoruz:

```
#include <QApplication>
#include <QWidget>
#include <QVBoxLayout>
#include <QPushButton>
#include <QLabel>
#include <QMessageBox>
#include <QInputDialog>
#include <QDialog>
#include <QFormLayout>
#include <QLineEdit>
#include <QDialogButtonBox>
#include <QDebug>

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    QWidget pencere;
    pencere.setWindowTitle("OBYS - Diyalog Ornekleri");
    pencere.resize(400, 300);

    auto* duzen = new QVBoxLayout(&pencere);
    auto* bilgiLabel = new QLabel("Bir butona tiklayin");
    bilgiLabel->setAlignment(Qt::AlignCenter);
    duzen->addWidget(bilgiLabel);
```

`QMessageBox` ile farklı türde mesaj diyalogları gösteriyoruz:

```
    // --- QMessageBox Ornekleri ---
    auto* bilgiButon = new QPushButton("Bilgi Mesaji");
    duzen->addWidget(bilgiButon);
    QObject::connect(bilgiButon, &QPushButton::clicked,
        [&pencere]() {
            QMessageBox::information(&pencere, "Bilgi",
                "Kayit i lemi basarili!");
        });

    auto* onayButon = new QPushButton("Onay Diyalogu");
    duzen->addWidget(onayButon);
    QObject::connect(onayButon, &QPushButton::clicked,
        [&pencere, bilgiLabel]() {
            auto cevap = QMessageBox::question(&pencere,
                "Onay", "Kaydi silmek istiyor musunuz?",
                QMessageBox::Yes | QMessageBox::No);
```

```
            if (cevap == QMessageBox::Yes)
                bilgiLabel->setText("Kayit silindi!");
            else
                bilgiLabel->setText("Silme iptal edildi.");
        });
```

Şimdi özel bir `QDialog` oluşturuyoruz:

```
    // --- zel QDialog ---
    auto* ozelButon = new QPushButton("  renci  Bilgi Gir");
    duzen->addWidget(ozelButon);

    QObject::connect(ozelButon, &QPushButton::clicked,
        [&pencere, bilgiLabel]() {
            QDialog diyalog(&pencere);
            diyalog.setWindowTitle("  renci  Bilgi Girisi");

            auto* formDuzen = new QFormLayout(&diyalog);
            auto* adAlani = new QLineEdit;
            auto* noAlani = new QLineEdit;
            noAlani->setMaxLength(9);
            formDuzen->addRow("Ad Soyad:", adAlani);
            formDuzen->addRow("  renci  No:", noAlani);

            auto* butonlar = new QDialogButtonBox(
                QDialogButtonBox::Ok
                | QDialogButtonBox::Cancel);
            formDuzen->addRow(butonlar);

            QObject::connect(butonlar,
                &QDialogButtonBox::accepted,
                &diyalog, &QDialog::accept);
            QObject::connect(butonlar,
                &QDialogButtonBox::rejected,
                &diyalog, &QDialog::reject);

            if (diyalog.exec() == QDialog::Accepted) {
                bilgiLabel->setText(
                    QString("Girilen: %1 (%2)")
                    .arg(adAlani->text())
                    .arg(noAlani->text()));
            }
        });

    pencere.show();
    return app.exec();
}
```

`exec()` ve Dönüş Değeri

`QDialog::exec()` modal diyaloğu açar ve kullanıcı kapatana kadar bekler. Dönüş değeri `QDialog::Accepted` veya `QDialog::Rejected` olur. Bu değer ile kullanıcının OK veya Cancel tıkladığını anlarsınız. `QDialogButtonBox` standart butonları otomatik düzenler.

Görev 3.1 – QMessageBox Türleri (10 dk)

Farklı `QMessageBox` türlerini deneyin:

a) Bir `"Uyari"` butonu ekleyin. Tıklandığında `QMessageBox::warning()` ile uyarı gösterin.

b) Bir `"Kritik Hata"` butonu ekleyin. Tıklandığında `QMessageBox::critical()` ile hata gösterin.

c) Her mesaj kutusundan sonra `bilgiLabel`'ı gösterilen mesaj türüyle güncelleyin.

Görev 3.2 – Doğrulamalı Diyalog ve QFileDialog (20 dk)

Özel diyaloğu genişletin ve `QFileDialog` ekleyin:

a) Öğrenci bilgi diyaloğuna doğrulama ekleyin: OK tıklandığında ad ve numara alanları boşsa `QMessageBox::warning()` ile uyarı gösterin ve diyaloğu kapatmayın.

b) `QDialogButtonBox::accepted` sinyalini doğrudan `accept()` yerine bir lambda'ya bağlayın:

```
QObject::connect(butonlar,
    &QDialogButtonBox::accepted,
    [&diyalog, adAlani, noAlani]() {
        if (adAlani->text().isEmpty()) {
            QMessageBox::warning(&diyalog, "Hata",
                "Ad alani bos olamaz!");
            return;
        }
        diyalog.accept();
    });
```

c) Yeni bir `"Dosya Sec"` butonu ekleyin. Tıklandığında `QFileDialog::getOpenFileName()` ile dosya seçtirin:

```
#include <QFileDialog>
QString dosya = QFileDialog::getOpenFileName(
    &pencere, "Dosya Sec", "",
    "CSV Dosyalari (*.csv);;Tum Dosyalar (*)");
```

d) Seçilen dosya yolunu `bilgiLabel`'da gösterin.

Dikkat: Diyalog Bellek Yönetimi

`exec()` ile çalıştırılan modal diyaloglar genellikle yerel (stack) değişken olarak oluşturulur – fonksiyon bittiğinde otomatik yok edilirler. Modeless diyaloglar ise `new` ile oluşturulmalı ve `setAttribute(Qt::WA_DeleteOnClose)` ile kapatıldığında otomatik silinmesi sağlanmalıdır.

4 Çalışma 4: OBYS Ana Pencere (30 dk)

Ön Bilgi

Bu çalışmada önceki bölümlerde öğrendiklerinizi birleştirerek tam işlevsel bir OBYS ana penceresi oluşturacaksınız: `QMainWindow` ile menüler, araç çubuğu, durum çubuğu, düzen yöneticileri ile form ve diyalog pencereleri bir arada kullanılacak.

Aşağıdaki başlangıç kodunu `main.cpp` dosyasına yazın. Önce ana pencere iskeletini kuruyoruz:

```
#include <QApplication>
#include <QMainWindow>
#include <QMenuBar>
#include <QToolBar>
#include <QStatusBar>
#include <QAction>
#include <QVBoxLayout>
#include <QHBoxLayout>
#include <QFormLayout>
#include <QWidget>
#include <QLabel>
#include <QLineEdit>
#include <QComboBox>
#include <QPushButton>
#include <QTableWidget>
#include <QMessageBox>
#include <QHeaderView>
#include <QDebug>

int main(int argc, char* argv[]) {
    QApplication app(argc, argv);

    QMainWindow anaPencere;
    anaPencere.setWindowTitle("OBYS - Ana Pencere");
    anaPencere.resize(800, 600);
```

Merkezi widget içinde form ve tablo alanını oluşturuyoruz:

```
    // Merkezi widget ve ana duzen
    auto* merkez = new QWidget;
    auto* anaDuzen = new QVBoxLayout(merkez);

    // Baslik
    auto* baslik = new QLabel("  renci   Bilgi Yonetim Sistemi");
    baslik->setStyleSheet(
        "font-size: 18px; font-weight: bold;"
        " color: #008080; padding: 5px;");
    baslik->setAlignment(Qt::AlignCenter);
    anaDuzen->addWidget(baslik);

    // Arama formu
    auto* formDuzen = new QHBoxLayout;
    formDuzen->addWidget(new QLabel("Ara:"));
    auto* aramaAlani = new QLineEdit;
    aramaAlani->setPlaceholderText("Ad veya numara...");
    formDuzen->addWidget(aramaAlani);
    auto* araButon = new QPushButton("Ara");
    formDuzen->addWidget(araButon);
    anaDuzen->addLayout(formDuzen);
```

QTableWidget ile öğrenci tablosu ekliyoruz:

```
    //    renci    tablosu
    auto* tablo = new QTableWidget(3, 4);
    tablo->setHorizontalHeaderLabels(
        {"No", "Ad Soyad", "B  l  m", "Ortalama"});
    tablo->horizontalHeader()->setStretchLastSection(true);

    // Ornek veri
    tablo->setItem(0, 0, new QTableWidgetItem("101"));
    tablo->setItem(0, 1, new QTableWidgetItem("Ali Yilmaz"));
    tablo->setItem(0, 2, new QTableWidgetItem("Bilgisayar"));
    tablo->setItem(0, 3, new QTableWidgetItem("3.45"));
    tablo->setItem(1, 0, new QTableWidgetItem("102"));
    tablo->setItem(1, 1, new QTableWidgetItem("Ayse Kaya"));
    tablo->setItem(1, 2, new QTableWidgetItem("Yazilim"));
    tablo->setItem(1, 3, new QTableWidgetItem("3.80"));
    tablo->setItem(2, 0, new QTableWidgetItem("103"));
    tablo->setItem(2, 1, new QTableWidgetItem("Veli Ozturk"));
    tablo->setItem(2, 2, new QTableWidgetItem("Elektrik"));
    tablo->setItem(2, 3, new QTableWidgetItem("2.95"));
    anaDuzen->addWidget(tablo);

    anaPencere.setCentralWidget(merkez);
```

Menü çubuğu, araç çubuğu ve sinyal bağlantılarını ekliyoruz:

```
    // Menu cubugu
    auto* dosyaMenu =
        anaPencere.menuBar()->addMenu("&Dosya");
    auto* yeniAksiyon =
        new QAction("&Yeni    renci   ", &anaPencere);
    yeniAksiyon->setShortcut(QKeySequence("Ctrl+N"));
    dosyaMenu->addAction(yeniAksiyon);
    dosyaMenu->addSeparator();
    auto* cikisAksiyon =
        new QAction("C&ikis", &anaPencere);
    dosyaMenu->addAction(cikisAksiyon);

    // Arac cubugu
    auto* aracCubugu =
        anaPencere.addToolBar("Islemler");
    aracCubugu->addAction(yeniAksiyon);

    // Durum cubugu
    anaPencere.statusBar()->showMessage(
        "3    renci    kayitli", 5000);

    // Baglanti: Cikis
    QObject::connect(cikisAksiyon, &QAction::triggered,
                     &app, &QApplication::quit);

    anaPencere.show();
    return app.exec();
}
```

`QTableWidget` Temel Kullanımı

`QTableWidget(satir, sutun)` ile tablo oluşturulur. `setHorizontalHeaderLabels()` ile sütun başlıkları atanır. `setItem(satir, sutun, new QTableWidgetItem("deger"))` ile hücrelere veri yazılır. Daha ileri kullanım için `QTableView + QAbstractTableModel` tercih edilir.

Görev 4.1 – Dock Widget Ekleme (15 dk)

`QDockWidget` ile yan panel ekleyin:

a) Sağ tarafa bir `QDockWidget` ekleyin: `"Öğrenci Detaylari"`.

```
#include <QDockWidget>
auto* dock = new QDockWidget(
    "  renci   Detaylari", &anaPencere);
dock->setAllowedAreas(
    Qt::LeftDockWidgetArea | Qt::RightDockWidgetArea);
```

b) Dock widget'ın içine `QFormLayout` ile bir detay formu yerleştirin: Ad, Bölüm, Ortalama etiketleri.

c) Dock widget'ı ana pencereye ekleyin:

```
anaPencere.addDockWidget(
    Qt::RightDockWidgetArea, dock);
```

d) Görünüm menüsü oluşturup dock widget'ın göster/gizle aksiyonunu ekleyin:

```
auto* g r n mMenu =
    anaPencere.menuBar()->addMenu("&G r n m");
g r n mMenu->addAction(dock->toggleViewAction());
```

Görev 4.2 – Menü Aksiyonlarını Bağlama (15 dk)

Menü aksiyonlarını işlevsel hale getirin:

a) `"Yeni Öğrenci"` aksiyonuna tıklandığında bir `QDialog` açın (Çalışma 3'teki gibi). Diyalogda Ad Soyad ve Bölüm alanları olsun.

b) Diyalogdan dönen bilgiyi tabloya yeni satır olarak ekleyin:

```
int satir = tablo->rowCount();
tablo->insertRow(satir);
tablo->setItem(satir, 0,
    new QTableWidgetItem(
        QString::number(100 + satir + 1)));
tablo->setItem(satir, 1,
    new QTableWidgetItem(adAlani->text()));
```

c) Her ekleme sonrasında durum çubuğunu güncelleyin: `"Toplam: X öğrenci kayitli"`.

d) Arama butonuna tıklandığında tablodaki eşleşen satırları vurgulayın (`setBackground()` ile).

Teslim Bilgileri

Teslim edilecekler:

1. `çalışma1/` – Düzen yöneticileri (Görev 1.1–1.2)
2. `çalışma2/` – QMainWindow ve menüler (Görev 2.1–2.2)
3. `çalışma3/` – Diyalog pencereleri (Görev 3.1–3.2)
4. `çalışma4/` – OBYS ana pencere (Görev 4.1–4.2)

Her çalışma klasörü bir `CMakeLists.txt` ve `main.cpp` dosyası içermelidir.
Puanlama: Çalışma 1: 25p Çalışma 2: 25p Çalışma 3: 25p Çalışma 4: 25p